

Burnout__train

October 22, 2025

1 Task 1: Burnout Prediction

In this notebook I look at several factors associated with employee burnout and try to predict whether an employee will/is experiencing burnout.

```
[ ]: import sys

sys.path.append("/Users/jared.mcbride/Documents/Code Projects/SVU-AI/Unit 2:␣
↳Neural Networks Basics/")

import pandas as pd
import matplotlib.pyplot as plt
import numpy as np

import Neurons
```

```
[ ]: df = pd.read_csv("/Users/jared.mcbride/Documents/Code Projects/SVU-AI/Unit 2:␣
↳Neural Networks Basics/synthetic_employee_burnout.csv")

df.tail()
```

```
[ ]:
```

	Name	Age	Gender	JobRole	Experience	WorkHoursPerWeek	\
1995	Leo Brown	41	Female	Manager	4	63	
1996	Alex Brown	23	Female	HR	2	39	
1997	Nina Wang	31	Female	HR	10	39	
1998	Kate Lee	25	Male	HR	0	40	
1999	Lily Petrov	49	Female	Engineer	13	65	

	RemoteRatio	SatisfactionLevel	StressLevel	Burnout
1995	17	3.40	4	0
1996	20	4.67	9	0
1997	4	4.10	4	0
1998	57	2.11	4	0
1999	22	4.36	8	0

```
[ ]: df = pd.read_csv("/Users/jared.mcbride/Documents/Code Projects/SVU-AI/Unit 2:␣
↳Neural Networks Basics/synthetic_employee_burnout.csv")
```

```
del df["Name"]

df = pd.get_dummies(df, columns=["Gender", "JobRole"])

df.tail()
```

```
[ ]:      Age  Experience  WorkHoursPerWeek  RemoteRatio  SatisfactionLevel  \
1995   41           4           63           17           3.40
1996   23           2           39           20           4.67
1997   31          10           39            4           4.10
1998   25           0           40           57           2.11
1999   49          13           65           22           4.36

      StressLevel  Burnout  Gender_Female  Gender_Male  JobRole_Analyst  \
1995           4         0           True          False           False
1996           9         0           True          False           False
1997           4         0           True          False           False
1998           4         0          False           True           False
1999           8         0           True          False           False

      JobRole_Engineer  JobRole_HR  JobRole_Manager  JobRole_Sales
1995              False          False           True           False
1996              False           True           False           False
1997              False           True           False           False
1998              False           True           False           False
1999               True          False           False           False
```

```
[ ]: df = df.sample(frac = 1).reset_index(drop=True)

df.tail()

Y = df.pop("Burnout")
Y = Y.to_numpy()
Y = np.where(Y == 0, -1, 1)

X = df.to_numpy()
X
```

```
[ ]: array([[36, 15, 49, ..., True, False, False],
           [53, 31, 67, ..., False, False, False],
           [35, 1, 64, ..., False, False, False],
           ...,
           [57, 0, 57, ..., False, False, False],
           [26, 0, 42, ..., False, True, False],
           [45, 24, 36, ..., False, False, True]], dtype=object)
```

```
[ ]: np.unique(Y)
```

```
[ ]: array([-1,  1])
```

```
[ ]: X_train = X[:1400,:]  
     X_test  = X[1400:,:]

     Y_train = Y[:1400]  
     Y_test  = Y[1400:]

     Y_test.shape
```

```
[ ]: (600,)
```

```
[ ]: ada = Neurons.Adaline(eta = 0.000_000_1, n_epochs = 100)  
     ada.fit(X_train, Y_train)

     ada.cost_
```

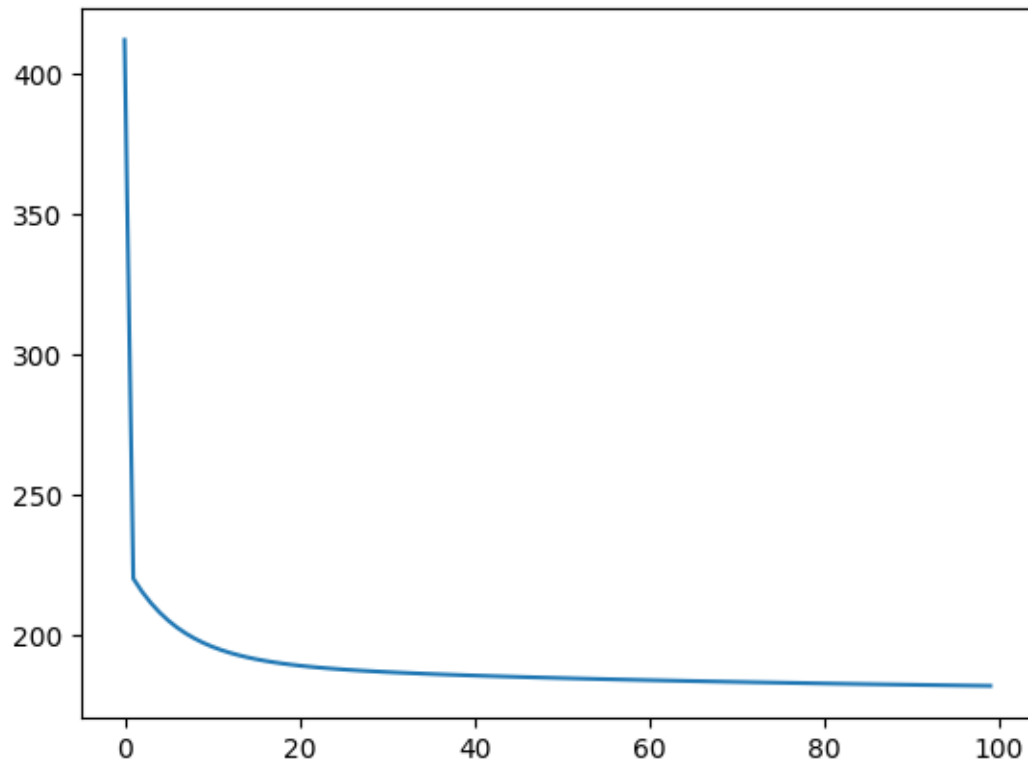
```
[ ]: [412.08618252089076,  
     220.15958993379152,  
     215.46966673900624,  
     211.48289081837666,  
     208.08805589474798,  
     205.1933645920303,  
     202.72136665179144,  
     200.6067123256205,  
     198.79425776489327,  
     197.23746722732923,  
     195.89706559238917,  
     194.73990196514458,  
     193.7379912939628,  
     192.8677061099653,  
     192.10909486678582,  
     191.44530704482509,  
     190.86210829235526,  
     190.34747149692603,  
     189.89123189092405,  
     189.48479615915326,  
     189.1208970882419,  
     188.79338662331435,  
     188.4970613152559,  
     188.22751508463134,  
     187.98101502333148,  
     187.75439662545298,  
     187.5449754043046,  
     187.35047232920596,  
     187.16895091784738,  
     186.99876415904797,
```

186.83850972667202,
186.68699218666106,
186.5431911024228,
186.4062341153872,
186.27537422209434,
186.14997059119477,
186.02947236656308,
185.9134049894739,
185.8013586459468,
185.69297850703427,
185.5879564818654,
185.48602424712828,
185.38694735366226,
185.2905202420484,
185.1965620253926,
185.1049129196908,
185.0154312208784,
184.92799074345726,
184.84247864890045,
184.75879360326454,
184.67684421290562,
184.59654769518903,
184.51782874780454,
184.44061858599525,
184.364854121793,
184.29047726339016,
184.21743431619586,
184.145675469999,
184.07515435907368,
184.00582768413676,
183.93765488675072,
183.87059786827615,
183.80462074665076,
183.7396896453538,
183.67577250976404,
183.6128389468663,
183.55086008487993,
183.48980844992306,
183.42965785724672,
183.37038331497135,
183.31196093855385,
183.25436787450326,
183.1975822320698,
183.1415830218236,
183.08635010022485,
183.03186411938654,
182.97810648137568,

```
182.92505929648706,  
182.8727053449961,  
182.8210280419864,  
182.77001140489062,  
182.71964002344006,  
182.66989903176264,  
182.62077408240162,  
182.5722513220563,  
182.52431736888042,  
182.47695929118652,  
182.43016458742855,  
182.38392116734875,  
182.33821733419757,  
182.29304176792948,  
182.2483835093002,  
182.20423194480753,  
182.1605767923957,  
182.11740808789082,  
182.0747161721019,  
182.03249167855196,  
181.9907255217962,  
181.94940888629438,  
181.9085332158008]
```

```
[ ]: plt.plot(ada.cost_)
```

```
[ ]: [<matplotlib.lines.Line2D at 0x14ca93ed0>]
```



```
[ ]: error_rate = np.mean(Y_test != ada.predict(X_test))
      error_rate
```

```
[ ]: 0.065
```

```
[ ]: burnout_rate = sum(Y_test == 1)/sum(Y_test == -1)
      burnout_rate
```

```
[ ]: 0.06951871657754011
```

1.1 Standardize

```
[ ]: mu = X_train.mean(axis=0)
      # For some reason, np.std(X_train, axis=0) gives an error here
      sig = np.array([np.std(X_train[:,i]) for i in range(X_train.shape[1])])

      X_train_std = (X_train - mu) / sig
      X_test_std = (X_test - mu) / sig
```

```
[ ]: adas = Neurons.Adaline(eta = 0.0001, n_epochs = 100)
      adas.fit(X_train_std, Y_train)
```

```
adas.cost_
```

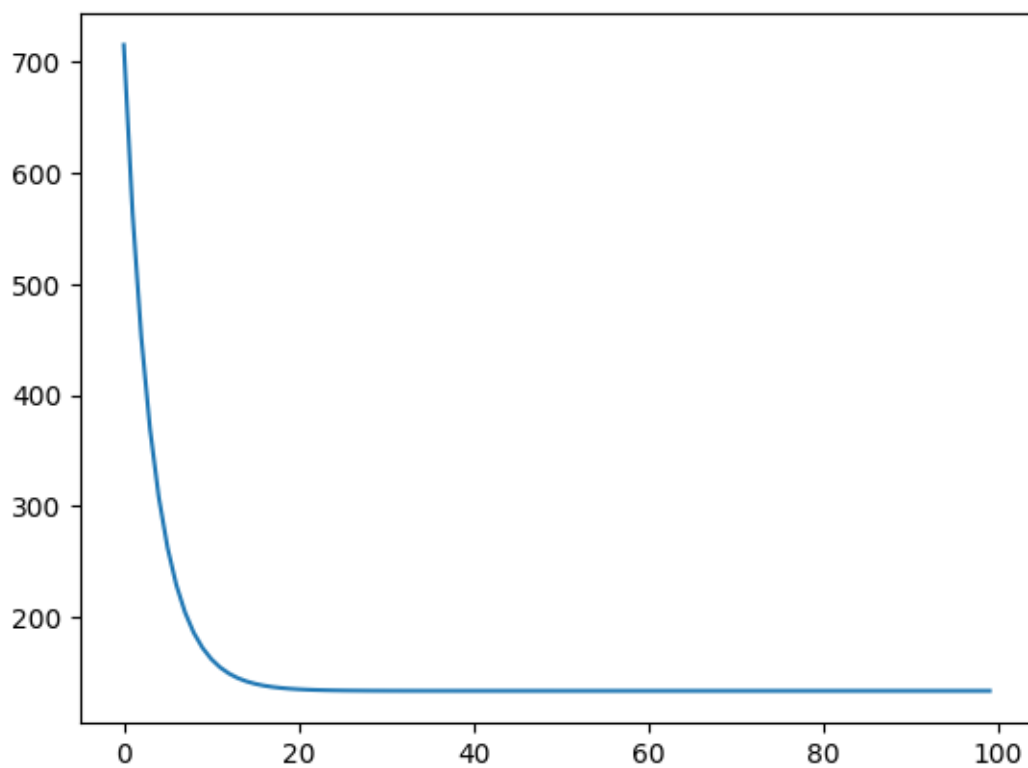
```
[ ]: [715.5948987884218,  
      564.0638335385717,  
      451.9912259411093,  
      369.09233061720096,  
      307.7669383446721,  
      262.39702535268407,  
      228.82888383084602,  
      203.99094380474153,  
      185.6115557038526,  
      172.01049356386864,  
      161.94487021980044,  
      154.49523615732846,  
      148.98136669791538,  
      144.89999016248342,  
      141.87873512258042,  
      139.64206914898836,  
      137.98610463276265,  
      136.7599620569886,  
      135.85198312942552,  
      135.17953112606753,  
      134.68144470086222,  
      134.31245458570174,  
      134.03905240876563,  
      133.83643382338795,  
      133.6862364742351,  
      133.57486605694726,  
      133.49225752229353,  
      133.43095826803582,  
      133.38544959751323,  
      133.3516444998912,  
      133.32651591706443,  
      133.30782158111663,  
      133.29390032473808,  
      133.28352129185677,  
      133.27577230370957,  
      133.26997720810996,  
      133.26563468332515,  
      133.26237292433368,  
      133.2599160870601,  
      133.2580594376687,  
      133.25665094705056,  
      133.25557765757543,  
      133.25475558363385,  
      133.25412222908056,  
      133.25363104272392,
```

133.25324730923396,
133.25294510327925,
133.252705031301,
133.25251255681738,
133.25235675810777,
133.2522294063153,
133.25212428104848,
133.25203666203996,
133.25196295135632,
133.2519003924308,
133.25184686093752,
133.25180070897272,
133.25176064882538,
133.251725666152,
133.25169495500774,
133.2516678691347,
133.25164388535217,
133.2516225759661,
133.25160358790606,
133.251586626889,
133.25157144534415,
133.25155783316185,
133.25154561055814,
133.2515346225425,
133.25152473459192,
133.25151582924155,
133.25150780337748,
133.25150056606572,
133.2514940367921,
133.25148814402567,
133.25148282403302,
133.25147801988663,
133.251473680636,
133.25146976059906,
133.2514662187594,
133.25146301824523,
133.25146012587732,
133.2514575117744,
133.251455149008,
133.2514530132943,
133.25145108272758,
133.25144933753697,
133.2514477598736,
133.25144633362046,
133.25144504421874,
133.25144387851756,
133.25144282463512,


```
133.2514418718332,  
133.2514410104098,  
133.25144023159407,  
133.2514395274604,  
133.25143889084376,  
133.251438315267,  
133.25143779487635,  
133.25143732437877]
```

```
[ ]: plt.plot(adas.cost_)
```

```
[ ]: [
```



```
[ ]: error_rate_std = np.mean(Y_test != adas.predict(X_test_std))  
error_rate_std
```

```
[ ]: 0.065
```

```
[ ]: Y = np.where(Y == -1, 0, 1)  
np.unique(Y)  
  
np.bincount(Y)/len(Y)
```

```
[ ]: array([0.9355, 0.0645])
```

2 PCA

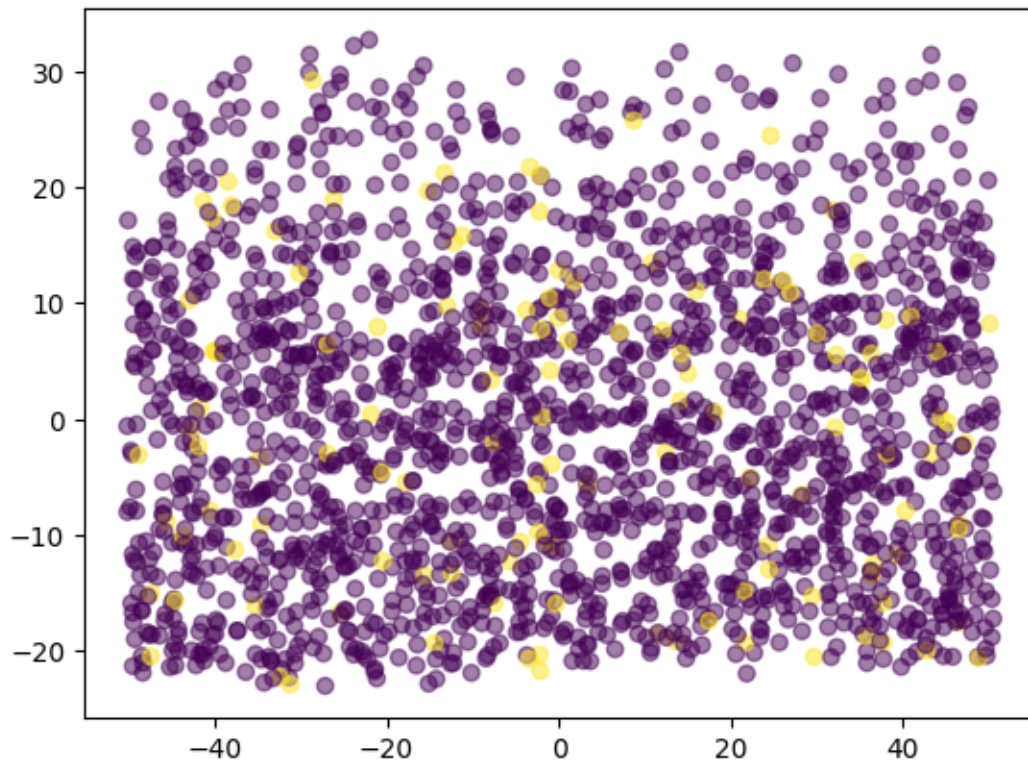
```
[ ]: from sklearn.decomposition import PCA
      from mpl_toolkits.mplot3d import Axes3D

      pca = PCA(n_components=3)
      X_reduced = pca.fit_transform(X)
      X_reduced.shape
```

```
[ ]: (2000, 3)
```

```
[ ]: plt.scatter(X_reduced[:,0], X_reduced[:,1], c=Y, cmap='viridis', alpha=0.5)
```

```
[ ]: <matplotlib.collections.PathCollection at 0x15418ce50>
```



```
[ ]: # Create a 3D scatter plot
      fig = plt.figure()
      ax = fig.add_subplot(111, projection='3d')

      # Scatter plot of the first three principal components
```

```

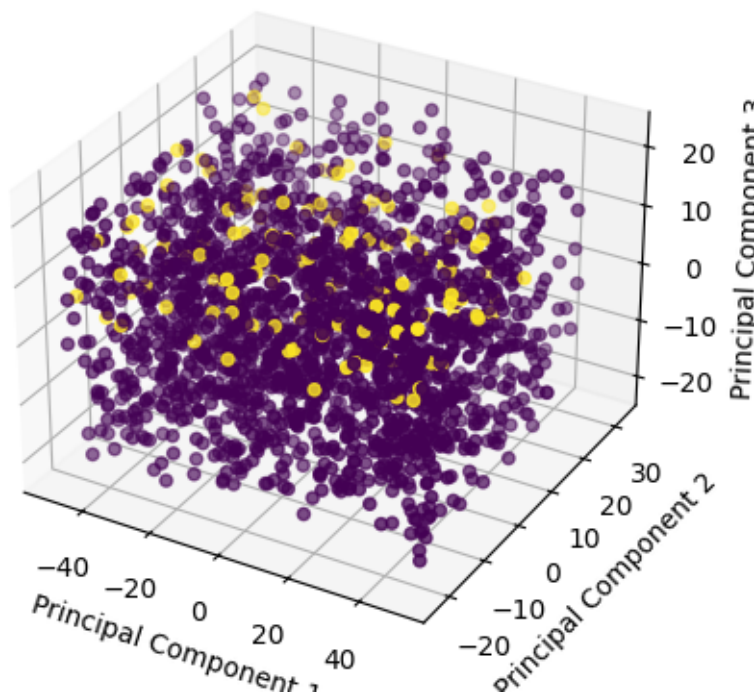
ax.scatter(X_reduced[:, 0], X_reduced[:, 1], X_reduced[:, 2], c=Y, marker='o')

# Set axis labels
ax.set_xlabel('Principal Component 1')
ax.set_ylabel('Principal Component 2')
ax.set_zlabel('Principal Component 3')

# Show the plot
plt.title("3D Scatter Plot of First Three Principal Components")
plt.show()

```

3D Scatter Plot of First Three Principal Components



```
[ ]: from sklearn.linear_model import LogisticRegression
```

```
[ ]: Y_train_m = np.where(Y_train == -1, 0, 1)
      Y_test_m = np.where(Y_test == -1, 0, 1)

      model=LogisticRegression(class_weight="balanced")
      model.fit(X_train_std,Y_train_m)
```

```
[ ]: LogisticRegression(class_weight='balanced')
```

```
[ ]: error_rate_lr = np.mean(Y_test_m != model.predict(X_test_std))  
error_rate_lr
```

```
[ ]: 0.065
```

```
[ ]: burnout_rate = sum(Y_test == 1)/sum(Y_test == -1)  
burnout_rate
```

```
[ ]: 0.06951871657754011
```

```
[ ]: X_train_std.mean(axis=0)
```

```
[ ]: array([-7.454354593911765e-18, -1.0142680346396966e-16,  
          3.612189912262563e-17,  9.706521301008512e-17,  
          2.1545860333966565e-15, -6.042785319745495e-17,  
          -9.563778340699563e-17,  9.563778340699563e-17,  
          1.8770699280626753e-16,  1.019581244828974e-15,  
          2.9658815086414894e-17, -3.9603241321271656e-16,  
          1.5860328923216522e-16], dtype=object)
```

```
[ ]:
```